

# Table of Contents

## Business Requirements Document (BRD)

### Restaurant Management System (RMS)

**Document version:** 1.0 **Date:** 7 July 2026 **Prepared by:** Project Team **Status:** Draft for review

---

#### 1. Document Purpose

This Business Requirements Document defines the business needs, scope, stakeholders, and high-level requirements for the Restaurant Management System (RMS). It establishes a shared understanding of *what* the system must achieve and *why*, before any design or coding begins. It serves as the reference against which the design, user stories, and delivered software are validated.

This document deliberately avoids implementation detail. Technology and design decisions are captured in the companion Development Plan.

---

#### 2. Executive Summary

The RMS is a software system that manages the core day-to-day operations of a restaurant: taking orders, producing bills and receipts, managing the menu and dining tables, tracking inventory and supplier purchases, handling reservations, and administering staff accounts. Management-level reporting sits on top of these operations to give owners visibility into sales, stock, and staff activity.

The system will be delivered in two stages. Stage one is a **desktop application** used on-premise by restaurant staff. Stage two migrates the same system to a **web-based application** accessible through a browser. To make this migration smooth, the system is designed from the start around a clean separation of concerns (the Model-View-Controller pattern) and a shared MySQL database, so that the business logic and data layer can be reused when the user interface moves to the web.

---

#### 3. Business Objectives

The project exists to achieve the following measurable business outcomes:

**BO-1 — Speed up service.** Reduce the time to take an order and produce a bill compared with a manual paper-based process, so tables turn over faster and queues shorten.

**BO-2 — Reduce billing errors.** Eliminate manual arithmetic mistakes by calculating item totals, taxes, discounts, and final bills automatically.

**BO-3 — Give management visibility.** Provide accurate sales, inventory, and staff reports on demand rather than relying on manual end-of-day tallies.

**BO-4 — Control stock and cost.** Track ingredient and stock levels against sales so that shortages and over-ordering are caught early.

**BO-5 — Centralise data.** Keep menu, pricing, orders, stock, staff, and reservation data in one consistent database rather than scattered notebooks or spreadsheets.

**BO-6 — Prepare for growth.** Deliver a desktop system now while keeping a clear, low-cost path to a multi-location, web-accessible system later.

---

## 4. Project Scope

### 4.1 In Scope

The system covers four functional areas plus administration and reporting:

1. **Orders & Billing (POS).** Creating orders for dine-in and takeaway, adding and removing menu items, applying discounts, calculating tax, splitting or finalising a bill, recording payment method, and printing/generating a receipt.
2. **Menu & Table Management.** Maintaining menu categories, menu items, prices and availability; defining the restaurant's tables and viewing/updating each table's status (free, occupied, reserved, needs cleaning).
3. **Inventory & Suppliers.** Recording stock items and quantities, registering suppliers, creating purchase orders, receiving stock, and flagging items that fall below a reorder level.
4. **Staff & Reservations.** Managing employee accounts and roles; recording customer reservations against a date, time, and table.
5. **Administration.** User accounts, roles and permissions, authentication (login/logout), and system reference data.
6. **Reporting.** Sales reports, inventory/stock reports, and staff activity reports for management.

### 4.2 Out of Scope (for the current phases)

The following are explicitly excluded from the initial desktop build and the first web migration, and may be considered in future phases: online customer-facing ordering and delivery, third-party payment gateway/card processing integration, kitchen display screens, loyalty/rewards programs, accounting/payroll integration, multi-branch consolidation, and a mobile app. Excluding these keeps the first delivery focused and achievable.

### 4.3 Phasing

- **Phase 1 — Desktop application** (JavaFX front end, MySQL database, MVC architecture). Full functionality for the four areas above, used on the restaurant premises.

- **Phase 2 — Web application.** The same business logic and database, exposed through a browser-based interface, enabling access from more devices and laying the groundwork for future multi-location use.
- 

## 5. Stakeholders

| Stakeholder                               | Interest in the system                                                                     |
|-------------------------------------------|--------------------------------------------------------------------------------------------|
| Restaurant Owner                          | Wants profitability, reliable reporting, and low operating friction. Sponsors the project. |
| Restaurant Manager                        | Runs daily operations; needs reports, menu control, staff and stock oversight.             |
| Cashier                                   | Front-line user taking orders and processing bills; needs speed and simplicity.            |
| Kitchen / Floor staff ( <i>indirect</i> ) | Affected by order flow and table status accuracy.                                          |
| Suppliers ( <i>external</i> )             | Provide stock recorded through purchase orders.                                            |
| Customers ( <i>external</i> )             | Experience faster service and accurate bills; may make reservations.                       |
| Development Team                          | Builds, tests, and migrates the system.                                                    |

## 6. User Roles and Permissions

The system defines three application roles. Permissions are cumulative in practice but defined explicitly to keep responsibilities clear.

**Administrator.** Full control of the system. Manages user accounts and roles, configures reference/system data, and can access every module and report. Typically the owner or an IT-responsible person.

**Manager.** Operational control. Manages the menu and prices, tables, inventory and suppliers, reservations, and staff records; runs all reports. Cannot manage system-level user accounts unless also granted Administrator rights.

**Cashier.** Operational front-line use. Creates and manages orders, produces bills and receipts, records payments, views the menu and table status, and records reservations. Cannot change prices, manage stock, or view management reports.

Access to each function is governed by role, and every user must authenticate before use.

---

## 7. Business Process Overview

The following describes the main operational flows the system supports.

**Order-to-payment flow.** A cashier selects a table (or marks the order as takeaway), adds menu items to the order, and sends it. Items and quantities accumulate on the order. When the customer is ready to pay, the system calculates the subtotal, applies any discount, adds tax, and produces the final bill. The cashier records the payment method, the order is closed, a receipt is generated, and the table is released.

**Menu maintenance flow.** A manager adds or edits menu categories and items, sets prices, and marks items available or unavailable. Changes are immediately reflected in the ordering screens.

**Inventory and purchasing flow.** A manager registers suppliers and stock items with reorder levels. When stock runs low, the manager raises a purchase order to a supplier and, on delivery, records the received quantity, which increases stock on hand.

**Reservation flow.** A cashier or manager records a reservation for a customer against a date, time, party size, and table. The table status reflects the reservation, and the reservation can be seated, completed, or cancelled.

**Reporting flow.** A manager or administrator selects a report and date range; the system aggregates the underlying data and presents sales, inventory, or staff activity results.

---

## 8. Functional Requirements

Functional requirements are grouped by module. Each has a unique identifier for traceability to user stories and test cases.

### 8.1 Authentication & Administration

- **FR-01** The system shall require every user to log in with a username and password before accessing any function.
- **FR-02** The system shall restrict access to functions based on the logged-in user's role.
- **FR-03** An administrator shall be able to create, edit, deactivate, and delete user accounts and assign roles.
- **FR-04** The system shall allow a user to log out and shall end the session on logout.

### 8.2 Menu & Table Management

- **FR-05** A manager shall be able to create, edit, and delete menu categories.
- **FR-06** A manager shall be able to create, edit, and delete menu items with a name, category, price, and availability status.
- **FR-07** The system shall allow menu items to be marked available or unavailable without deleting them.
- **FR-08** A manager shall be able to define dining tables with a number/label and seating capacity.
- **FR-09** The system shall display and update each table's status (free, occupied, reserved, needs cleaning).

### 8.3 Orders & Billing (POS)

- **FR-10** A cashier shall be able to open a new order for a specific table or as takeaway.

- **FR-11** A cashier shall be able to add menu items and quantities to an open order and remove them before finalising.
- **FR-12** The system shall calculate the order subtotal automatically from item prices and quantities.
- **FR-13** The system shall support applying a discount to an order.
- **FR-14** The system shall calculate tax and the final total automatically.
- **FR-15** A cashier shall be able to record the payment method and finalise (close) the order.
- **FR-16** The system shall generate a printable/exportable receipt for a finalised order.
- **FR-17** The system shall release the table (set to needs cleaning/free) when a dine-in order is closed.

#### 8.4 Inventory & Suppliers

- **FR-18** A manager shall be able to create, edit, and delete stock items with a unit of measure and reorder level.
- **FR-19** A manager shall be able to register and maintain suppliers.
- **FR-20** A manager shall be able to create a purchase order for a supplier listing items and quantities.
- **FR-21** The system shall increase stock on hand when a purchase order's delivery is recorded as received.
- **FR-22** The system shall flag stock items whose quantity is at or below the reorder level.

#### 8.5 Staff & Reservations

- **FR-23** A manager shall be able to create, edit, and deactivate staff records (name, role, contact, status).
- **FR-24** A cashier or manager shall be able to create a reservation with customer name, contact, date, time, party size, and table.
- **FR-25** The system shall allow a reservation to be seated, completed, or cancelled.
- **FR-26** The system shall prevent double-booking the same table for overlapping reservation times.

#### 8.6 Reporting

- **FR-27** A manager shall be able to generate a sales report for a chosen date range showing totals and item breakdowns.
  - **FR-28** A manager shall be able to generate an inventory report showing current stock and items below reorder level.
  - **FR-29** A manager shall be able to generate a staff activity report.
  - **FR-30** The system shall allow reports to be exported (e.g., to PDF or a printable format).
- 

### 9. Non-Functional Requirements

**NFR-01 — Usability.** Front-line ordering and billing screens shall be simple enough for a cashier to learn in a single short training session, with minimal clicks for common actions.

**NFR-02 — Performance.** Common actions (opening an order, adding an item, generating a bill) shall respond within about two seconds under normal single-restaurant load.

**NFR-03 — Reliability & data integrity.** The system shall use database constraints and transactions so that financial and stock data remain consistent; a failed operation shall not leave partial data.

**NFR-04 — Security.** Passwords shall be stored securely (hashed, not plain text). Access shall be enforced by role. Financial and personal data shall not be exposed to unauthorised roles.

**NFR-05 — Maintainability.** The system shall follow the MVC pattern, separating data, business logic, and presentation, so that the interface can be replaced (desktop to web) without rewriting business logic.

**NFR-06 — Portability / migration readiness.** Business logic and data access shall be kept independent of the desktop UI framework so they can be reused by the future web interface against the same MySQL database.

**NFR-07 — Data persistence.** All operational data shall be stored in a MySQL relational database normalised to at least third normal form (3NF).

**NFR-08 — Availability.** For the desktop phase, the system shall function on the restaurant's local machine/network without requiring internet access.

---

## 10. Data Requirements (High Level)

The system will store and relate, at minimum, the following core entities: **Users/Staff, Roles, Menu Categories, Menu Items, Tables, Orders, Order Items** (the line items of an order), **Payments, Stock Items, Suppliers, Purchase Orders, Purchase Order Items**, and **Reservations**. Detailed attributes and relationships are defined in the conceptual and logical database design during the design phase, and must normalise to 3NF. The database is shared across both the desktop and future web phases.

---

## 11. Assumptions

- The restaurant operates as a single location for the initial phases.
  - A reliable local computer (and network, if multi-terminal) is available for the desktop phase.
  - Payment is recorded in the system but processed outside it (no card gateway integration in scope).
  - Menu, tax rate, and reference data are provided by the restaurant during setup.
  - Staff using the cashier role have basic computer familiarity.
-

## 12. Constraints

- **Technology:** Desktop client built with Java/JavaFX; data stored in MySQL; architecture follows MVC.
  - **Migration:** The design must not lock business logic into the desktop UI, so that a web front end can reuse it.
  - **Budget/effort:** Built by a small team as a project, favouring proven, well-documented technologies over experimental ones.
- 

## 13. Risks (Summary)

| Risk                                   | Impact                      | Mitigation                                                 |
|----------------------------------------|-----------------------------|------------------------------------------------------------|
| Scope creep from out-of-scope features | Delays delivery             | Hold the phased scope; log extras as future phases         |
| Desktop logic too coupled to UI        | Costly web migration        | Enforce MVC and a service/data layer independent of JavaFX |
| Data model changes late                | Rework across modules       | Finalise and normalise the schema before heavy coding      |
| Incorrect financial calculations       | Loss of trust, money errors | Centralise billing logic; test thoroughly with edge cases  |

## 14. Success Criteria

The project is successful when: a cashier can complete an order-to-receipt cycle entirely in the system; a manager can maintain the menu, stock, suppliers, and reservations and generate accurate reports; all data persists correctly in a 3NF MySQL database; access is correctly governed by role; and the architecture allows the web front end to be added in Phase 2 without rewriting the business and data layers.

---

## 15. Approval

Role

---

Project Sponsor / Owner

Manager (Business Owner of requirements)

Development Lead